

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem Solving Task

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA1516
---------------------	--	---------------------	---------

Course Outcome Covered

CO4: Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)

Assessment Tool Weightage: 20%

Code of Conduct

I AMARA SATWIK- **192525204** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.SATWIK

Assessment Tasks — Industry Problem Solving Task

Dave's Taxonomy: Articulation (Level 4) | Question Count: 5 | Total Marks: 40

Q1. An e-commerce company receives clickstream, transaction, and review data from millions of users daily. Propose a big data analytics pipeline and explain how the 5Vs influence architecture design.

Proposed Pipeline

- **1. Data Ingestion Layer:** Clickstream events and transactions are captured in real time using a distributed message queue such as Apache Kafka, while batch data (e.g., daily order exports, review dumps) is ingested via Sqoop/Flume or scheduled ETL jobs.
- **2. Storage Layer:** Raw and processed data is stored in a distributed data lake (HDFS or cloud object storage such as Amazon S3), partitioned by date/event type for efficient retrieval. Structured transaction summaries can additionally be loaded into a data warehouse (e.g., Amazon Redshift/Snowflake) for BI reporting.
- **3. Processing Layer:** Apache Spark (batch) handles large-scale aggregation of historical clickstream and review data, while Spark Streaming or Kafka Streams performs real-time processing for tasks like live recommendation updates and fraud checks.
- **4. Analytics & ML Layer:** Recommendation engines, sentiment analysis on reviews (NLP), and customer segmentation models run on the processed data, typically using Spark MLlib or a dedicated ML platform.
- **5. Serving/Visualization Layer:** Results are pushed to dashboards (Power BI/Tableau) for business teams and to low-latency serving stores (e.g., Redis, Cassandra) for real-time features like product recommendations shown on the website.

How the 5Vs Influence Architecture Design

- **Volume:** Millions of daily events across clickstream, transactions, and reviews demand horizontally scalable, distributed storage (HDFS/S3) and processing (Spark) rather than a single relational database.
- **Velocity:** Clickstream and transaction data arrive continuously and must be processed with low latency (e.g., real-time fraud detection, live inventory updates), which drives the choice of a streaming layer such as Kafka + Spark Streaming instead of purely batch ETL.
- **Variety:** Clickstream logs (semi-structured JSON), transactions (structured), and reviews (unstructured text) require a flexible schema-on-read data lake architecture that can store all three types together, rather than a rigid relational schema.
- **Veracity:** Review data and clickstream logs can be noisy (bot traffic, duplicate events, fake reviews), so the pipeline must include data-cleaning/validation stages and anomaly filtering before analytics is trusted for decisions.
- **Value:** The architecture is designed around extracting specific business value — e.g., personalized recommendations and demand forecasting — so processing and storage choices prioritize the data streams that feed these high-value use cases.

Q2. A healthcare organization collects structured patient records, semi-structured insurance claims, and unstructured medical images. Design an approach to manage and secure this data ecosystem.

Data Management Approach

- **Structured patient records (EHR/demographics):** Stored in a relational database or a structured zone of a data lakehouse (e.g., using Delta Lake/Apache Hudi) so ACID transactions and easy SQL querying remain possible for clinical workflows.
- **Semi-structured insurance claims (XML/JSON/HL7 messages):** Stored in a NoSQL document store (e.g., MongoDB) or as structured files in the data lake, using a consistent schema-on-read approach so nested claim fields (diagnosis codes, billing details) can be parsed as needed.
- **Unstructured medical images (X-rays, MRIs, scans):** Stored in a distributed object store or DFS (e.g., HDFS/S3) with metadata (patient ID, scan type, date) indexed separately in a searchable catalog, enabling image-processing/ML pipelines (e.g., diagnostic AI models) to access them efficiently.
- **Unified access layer:** A data catalog / metadata layer links records across the three types (e.g., a common patient ID) so clinicians and analytics tools can query a unified 360° view of the patient without merging the underlying storage systems.

Security Approach

- **Encryption at rest and in transit:** All patient data, claims, and images must be encrypted using strong standards (e.g., AES-256) both while stored and while moving between systems, to protect against interception or storage breaches.
- **Role-based access control (RBAC) and least privilege:** Only authorized clinical, billing, or research staff should access specific data categories — e.g., billing staff should not need direct access to raw medical images.
- **De-identification / anonymization:** For analytics or research use cases, personally identifiable information (PII) should be masked or removed (following HIPAA/regional data-protection norms) to reduce re-identification risk.
- **Audit logging and monitoring:** Every access or modification to patient data should be logged and monitored to detect unauthorized access and support compliance audits.

- **Regulatory compliance:** The overall design should align with healthcare data regulations (e.g., HIPAA, or India's DPDP Act) covering consent management, data retention, and breach-notification requirements.

Q3. A telecom company wants to detect churn patterns using high-volume customer interaction data. Identify the big data challenges and recommend suitable analytics and integration strategies.

Big Data Challenges

- **High volume and velocity of interaction data:** Call detail records (CDRs), app usage logs, customer-support chats, and network-quality metrics are generated continuously at massive scale, making it difficult to process and analyze in near real time using traditional systems.
- **Data variety and integration complexity:** Interaction data comes from many heterogeneous sources — structured billing systems, semi-structured CDRs, and unstructured support-call transcripts — requiring a unified pipeline to combine them meaningfully for a single customer view.
- **Data quality and veracity:** Missing, duplicate, or inconsistent records (e.g., dropped call logs, mismatched customer IDs across systems) can distort churn models if not cleaned properly.
- **Class imbalance and timely detection:** Churners are typically a small fraction of the customer base, and by the time churn is confirmed it is often too late to intervene, so the system must detect early warning signals rather than just historical churn.

Recommended Analytics & Integration Strategies

- Use a distributed ETL/integration tool (e.g., Apache NiFi or Talend) to ingest and standardize data from billing systems, network logs, and CRM into a common data lake, using Sqoop for periodic RDBMS-to-HDFS transfers.
- Apply Apache Spark for feature engineering (e.g., call-drop rate, complaint frequency, usage decline trend) at scale across millions of subscriber records.
- Build a machine learning churn-prediction model (e.g., gradient boosting or random forest) trained on historical churn labels, with retraining pipelines to adapt to changing customer behavior.
- Incorporate real-time streaming analytics (Kafka + Spark Streaming) to flag high-risk customers immediately after negative interactions (e.g., a dropped call followed by a complaint), enabling proactive retention offers.
- Use data quality/validation frameworks in the pipeline to catch missing or inconsistent records before they reach the model, improving prediction reliability.

Q4. A smart city project generates sensor streams from traffic, air quality, and surveillance systems. Explain how distributed file systems and data integration tools can support large-scale analysis.

Role of Distributed File Systems (e.g., HDFS)

- **Scalable storage for continuous sensor streams:** Traffic cameras, air-quality sensors, and surveillance feeds generate huge, ever-growing volumes of data; a DFS allows storage capacity to scale horizontally by simply adding more nodes to the cluster.
- **Fault tolerance for critical infrastructure data:** Sensor data replicated across multiple nodes ensures that hardware failure at any single node does not result in permanent data loss — important for safety-critical applications like traffic and surveillance monitoring.
- **Support for parallel batch analysis:** Historical sensor data stored in HDFS can be processed in parallel using frameworks like Spark or Hadoop MapReduce, enabling large-scale analysis such as identifying long-term traffic congestion or pollution trends across the city.

- Unified storage for heterogeneous data: Traffic sensor readings (structured), air-quality time series (semi-structured), and surveillance video (unstructured) can all be stored together in the same distributed file system, simplifying downstream analytics.

Role of Data Integration Tools

- Real-time ingestion: Tools like Apache NiFi or Kafka Connect collect streaming data from thousands of distributed sensors and route it reliably into the storage/processing pipeline, handling network interruptions and out-of-order data.
- Data standardization: Integration tools normalize different sensor data formats (e.g., varying units, timestamps, device protocols from different vendors) into a consistent schema for unified analysis.
- Cross-domain correlation: By integrating traffic, air-quality, and surveillance data streams into a common platform, city planners can correlate events (e.g., traffic congestion causing spikes in air pollution) that would be invisible if each system were analyzed in isolation.
- Feeding real-time dashboards and alerts: Integration pipelines can push processed data to real-time dashboards or trigger automated alerts (e.g., traffic-signal adjustment, pollution alerts) for immediate city-management action.

Q5. A financial analytics firm must comply with strict privacy requirements while processing high-velocity transaction data. Design a secure big data solution and justify the controls chosen.

Proposed Secure Architecture

- **Ingestion with encryption in transit:** Transaction data is streamed via a secure, encrypted channel (TLS) into a message broker such as Kafka, ensuring data cannot be intercepted while moving from source systems into the pipeline.
- **Tokenization/masking at ingestion:** Sensitive fields (account numbers, card numbers, personal identifiers) are tokenized or masked immediately upon ingestion, so raw sensitive values are never stored in downstream analytics systems.
- **Encrypted, access-controlled storage:** Processed data is stored in a distributed store (HDFS/cloud storage) with encryption at rest (AES-256) and strict role-based access control, ensuring only authorized analysts and systems can query specific data categories.
- **Real-time stream processing for fraud detection:** Spark Streaming/Kafka Streams analyzes high-velocity transaction data in real time to detect anomalies (e.g., unusual spending patterns) without needing to expose raw sensitive data to every downstream consumer.
- **Audit logging and monitoring:** Every access, query, and modification is logged centrally (e.g., using a SIEM tool) to support compliance audits and to quickly detect any unauthorized access attempt.

Justification of Controls

- Encryption (in transit and at rest) directly addresses confidentiality requirements mandated by financial regulations (e.g., RBI/PCI-DSS guidelines), preventing data exposure even if storage media or network traffic is compromised.
- Tokenization/masking limits the blast radius of any potential breach — even if an attacker accesses the analytics layer, they cannot recover raw sensitive account details.
- RBAC and least-privilege access ensure that only staff with a genuine business need (e.g., fraud analysts) can view sensitive transaction details, reducing insider-threat risk.
- Real-time processing balances the need for speed (fraud must be caught within seconds) against security, by analyzing tokenized/pseudonymized data rather than requiring raw data exposure at every stage.

- Audit logging supports regulatory compliance (demonstrating accountability and traceability) and enables rapid incident response if suspicious access patterns are detected.

— End of Submission —